

BatchNormalization

2026 年 6 月 10 日

1 批标准化 (Batch Normalization)

使深度网络更容易训练的一种方法是使用更复杂的优化程序，如 SGD+ 动量 (momentum)、RM-SProp 或 Adam。另一种策略是改变网络的架构，使其更易于训练。沿着这个思路的一个想法是批标准化，由 [1] 在 2015 年提出。

要理解批标准化的目标，首先需要认识到：机器学习方法在处理由零均值、单位方差且不相关的特征组成的输入数据时，往往表现更好。在训练神经网络时，我们可以在将数据输入网络之前对其进行预处理，以显式地消除特征之间的相关性。这将确保网络的第一层能够处理符合良好分布的数据。然而，即使我们对输入数据进行了预处理，网络深层的激活值也可能不再是不相关的，也不再具有零均值或单位方差，因为它们是从网络的较早层输出的。更糟糕的是，在训练过程中，随着每层权重的更新，网络每层特征的分布都会发生偏移。

[1] 的作者假设，深度神经网络内部特征分布的偏移可能会使深度网络的训练更加困难。为了克服这个问题，他们建议在网络中插入对批次进行标准化的层。在训练时，这样的层使用一小批数据来估计每个特征的均值和标准差。然后，这些估计的均值和标准差被用来对该批次的特征进行中心化和标准化。在训练过程中，会保持这些均值和标准差的移动平均值 (running average)，而在测试时，这些移动平均值被用来对特征进行中心化和标准化。

这种标准化策略可能会降低网络的表示能力，因为对于某些层来说，拥有非零均值或非单位方差的特征有时可能是最优的。为此，批标准化层包含了每个特征维度的可学习偏移 (shift) 和缩放 (scale) 参数。

[1] [Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”, ICML 2015.](#)

```
[14]: # 配置 matplotlib 中文字体
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False
```

```

# 配置单元格
import time # 导入时间模块, 用于计时等操作
import numpy as np # 导入 numpy 库, 用于数值计算
import matplotlib.pyplot as plt # 导入 matplotlib 库, 用于数据可视化
from cs231n.data_utils import get_CIFAR10_data # 从 cs231n 的 data_utils 模块导入获取 CIFAR-10 数据的函数
from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradient_array # 从 gradient_check 模块导入数值梯度计算函数
from cs231n.solver import Solver # 从 solver 模块导入 Solver 类, 用于训练模型

# 设置 matplotlib 在 notebook 中内嵌显示图像
%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0) # 设置图像的默认大小
plt.rcParams["image.interpolation"] = "nearest" # 设置图像插值方式为最近邻插值
plt.rcParams["image.cmap"] = "gray" # 设置默认的图像颜色映射为灰度

# 加载 autoreload 扩展, 用于自动重新加载修改过的模块
get_ipython().run_line_magic('load_ext', 'autoreload')
# 设置 autoreload 模式为 2, 即所有导入的模块都会被自动重新加载
get_ipython().run_line_magic('autoreload', '2')

def rel_error(x, y):
    """ 计算相对误差 """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))

def print_mean_std(x, axis=0):
    print(f" 均值: {x.mean(axis=axis)}")
    print(f" 标准差: {x.std(axis=axis)}\n")

```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```

[3]: # 加载 (预处理后的) CIFAR-10 数据集
data = get_CIFAR10_data() # 调用获取 CIFAR-10 数据的函数, 返回预处理后的数据集

# 遍历数据集中的所有项并打印它们的形状

```

```
for k, v in list(data.items()):
    print(f"{k}: {v.shape}") # 打印每个数据项的键（名称）及其对应的数组形状
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

2 批量归一化：前向传播

在文件 `cs231n/layers.py` 中，实现批量归一化的前向传播（forward pass），在函数 `batchnorm_forward` 中完成。

完成后，运行以下代码测试你的实现。

参考文献 [1] 中的论文或许会有所帮助！

```
[5]: from cs231n.layers import * # 从 cs231n.layers 模块导入所有函数/类

# 通过检查批标准化前后特征的均值和方差，来验证训练时的前向传播

# 模拟双层网络的前向传播过程
np.random.seed(231) # 设置随机种子，保证结果可复现
N, D1, D2, D3 = 200, 50, 60, 3 # 定义变量：N 是样本数，D1-D3 是各层维度
X = np.random.randn(N, D1) # 生成随机输入数据
W1 = np.random.randn(D1, D2) # 第一层权重
W2 = np.random.randn(D2, D3) # 第二层权重
a = np.maximum(0, X.dot(W1)).dot(W2) # 计算网络输出（ReLU 激活后再经过第二层）

print('批标准化之前：')
print_mean_std(a, axis=0) # 打印特征的均值和标准差（按特征维度计算）

gamma = np.ones((D3,)) # 初始化缩放参数 gamma 为全 1
beta = np.zeros((D3,)) # 初始化偏移参数 beta 为全 0

# 标准化后，均值应接近 0，标准差应接近 1
print('批标准化之后 (gamma=1, beta=0)')
```

```

a_norm, _ = batchnorm_forward(a, gamma, beta, {'mode': 'train'}) # 执行批标准化前向传播
print_mean_std(a_norm, axis=0) # 打印标准化后的均值和标准差

gamma = np.asarray([1.0, 2.0, 3.0]) # 自定义 gamma 值
beta = np.asarray([11.0, 12.0, 13.0]) # 自定义 beta 值

# 此时均值应接近 beta, 标准差应接近 gamma
print('批标准化之后 (gamma=', gamma, ', beta=', beta, ')')
a_norm, _ = batchnorm_forward(a, gamma, beta, {'mode': 'train'}) # 执行批标准化前向传播
print_mean_std(a_norm, axis=0) # 打印标准化后的均值和标准差

```

批标准化之前:

均值: [-2.3814598 -13.18038246 1.91780462]

标准差: [27.18502186 34.21455511 37.68611762]

批标准化之后 (gamma=1, beta=0)

均值: [8.88178420e-18 -3.10862447e-17 3.46944695e-19]

标准差: [0.99999999 1. 1.]

批标准化之后 (gamma= [1. 2. 3.] , beta= [11. 12. 13.])

均值: [11. 12. 13.]

标准差: [0.99999999 1.99999999 2.99999999]

[6]: # 通过多次运行训练时的前向传播来预热移动平均值,
然后检查测试时前向传播后的激活值的均值和方差, 以此验证测试时的前向传播

```

np.random.seed(231) # 设置随机种子, 确保结果可复现
N, D1, D2, D3 = 200, 50, 60, 3 # 定义变量: N 为样本数, D1-D3 为各层维度
W1 = np.random.randn(D1, D2) # 第一层权重
W2 = np.random.randn(D2, D3) # 第二层权重

bn_param = {'mode': 'train'} # 批标准化参数, 初始设为训练模式
gamma = np.ones(D3) # 缩放参数 gamma 初始化为全 1
beta = np.zeros(D3) # 偏移参数 beta 初始化为全 0

```

```

# 多次运行训练模式的前向传播，预热（更新）移动平均值
for t in range(50):
    X = np.random.randn(N, D1) # 生成随机输入数据
    a = np.maximum(0, X.dot(W1)).dot(W2) # 计算网络输出
    batchnorm_forward(a, gamma, beta, bn_param) # 执行批标准化前向传播（更新移动平均）

bn_param['mode'] = 'test' # 将批标准化模式切换为测试模式
X = np.random.randn(N, D1) # 生成新的随机输入数据
a = np.maximum(0, X.dot(W1)).dot(W2) # 计算网络输出
a_norm, _ = batchnorm_forward(a, gamma, beta, bn_param) # 执行测试模式下的批标准化前向传播

# 此时均值应接近 0，标准差应接近 1，但会比训练时的前向传播结果噪声更大
print('批标准化之后（测试时）：')
print_mean_std(a_norm, axis=0) # 打印测试时标准化后的均值和标准差

```

批标准化之后（测试时）：

均值：[-0.03927354 -0.04349152 -0.10452688]

标准差：[1.01531428 1.01238373 0.97819988]

3 批量归一化：反向传播

现在在函数 `batchnorm_backward` 中实现批量归一化的反向传播（backward pass）。

为了推导反向传播，你应该写出批量归一化的计算图，并通过每个中间节点进行反向传播。一些中间节点可能有多个输出分支；在反向传播中，确保将这些分支的梯度相加。参考文献 [1] 中的论文或许会有所帮助！

完成后，运行以下代码进行数值梯度检查。

```

[7]: # 梯度检验批标准化的反向传播
np.random.seed(231) # 设置随机种子，确保结果可复现
N, D = 4, 5 # N 为样本数，D 为特征维度
x = 5 * np.random.randn(N, D) + 12 # 生成随机输入数据（均值约为 12，标准差约为 5）
gamma = np.random.randn(D) # 随机初始化缩放参数 gamma

```

```

beta = np.random.randn(D) # 随机初始化偏移参数 beta
dout = np.random.randn(N, D) # 生成随机的输出梯度

bn_param = {'mode': 'train'} # 批标准化参数, 设置为训练模式

# 定义用于数值梯度计算的匿名函数
# fx: 以 x 为变量的批标准化前向传播函数
fx = lambda x: batchnorm_forward(x, gamma, beta, bn_param)[0]
# fg: 以 gamma 为变量的批标准化前向传播函数
fg = lambda a: batchnorm_forward(x, a, beta, bn_param)[0]
# fb: 以 beta 为变量的批标准化前向传播函数
fb = lambda b: batchnorm_forward(x, gamma, b, bn_param)[0]

# 使用数值方法计算梯度 (用于检验)
dx_num = eval_numerical_gradient_array(fx, x, dout) # x 的数值梯度
da_num = eval_numerical_gradient_array(fg, gamma.copy(), dout) # gamma 的数值梯度
db_num = eval_numerical_gradient_array(fb, beta.copy(), dout) # beta 的数值梯度

# 执行批标准化的前向传播和反向传播, 得到解析梯度
_, cache = batchnorm_forward(x, gamma, beta, bn_param)
dx, dgamma, dbeta = batchnorm_backward(dout, cache)

# 你应该会看到相对误差在  $1e-13$  到  $1e-8$  之间
print('dx 误差: ', rel_error(dx_num, dx))
print('dgamma 误差: ', rel_error(da_num, dgamma))
print('dbeta 误差: ', rel_error(db_num, dbeta))

```

```

dx 误差: 1.7029261167605239e-09
dgamma 误差: 7.420414216247087e-13
dbeta 误差: 2.8795057655839487e-12

```

4 批量归一化: 替代反向传播

在课堂上, 我们讨论了 Sigmoid 反向传播的两种不同实现方法。一种策略是写出由简单运算组成的计算图, 并对所有中间值进行反向传播。另一种策略是在纸上推导导数。例如, 你可以通过在纸上简化梯度, 为 Sigmoid 函数的反向传播推导出一个非常简单的公式。

令人惊讶的是，对于批量归一化的反向传播，你也可以进行类似的简化！

在前向传播中，给定一组输入 $X = \begin{bmatrix} x_1 \\ x_2 \\ \dots \\ x_N \end{bmatrix}$ ，

我们首先计算均值 μ 和方差 v 。有了 μ 和 v ，我们可以计算标准差 σ 和归一化数据 Y 。以下方程和图示描述了计算过程（ y_i 是向量 Y 的第 i 个元素）。

$$\mu = \frac{1}{N} \sum_{k=1}^N x_k \quad v = \frac{1}{N} \sum_{k=1}^N (x_k - \mu)^2 \quad (1)$$

$$\sigma = \sqrt{v + \epsilon} \quad y_i = \frac{x_i - \mu}{\sigma} \quad (2)$$

在反向传播过程中，我们问题的核心是：在已知接收的上游梯度 $\frac{\partial L}{\partial Y}$ 的情况下，计算 $\frac{\partial L}{\partial X}$ 。要做到这一点，回想一下微积分中的链式法则： $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial X}$ 。

未知/困难的部分是 $\frac{\partial Y}{\partial X}$ 的求解。我们可以通过以下方式推导：首先逐步得出局部梯度 $\frac{\partial v}{\partial X}$ 、 $\frac{\partial \mu}{\partial X}$ 、 $\frac{\partial \sigma}{\partial v}$ 、 $\frac{\partial Y}{\partial \sigma}$ 和 $\frac{\partial Y}{\partial \mu}$ ，然后利用链式法则适当地组合这些梯度（它们以向量形式存在！），最终计算出 $\frac{\partial Y}{\partial X}$ 。

如果直接推导涉及矩阵乘法的 X 和 Y 的梯度有难度，可以先从单个元素 x_i 和 y_i 的角度分析梯度：这种情况下，你需要先依靠链式法则计算中间梯度 $\frac{\partial \mu}{\partial x_i}$ 、 $\frac{\partial v}{\partial x_i}$ 、 $\frac{\partial \sigma}{\partial x_i}$ ，再组合这些结果推导出 $\frac{\partial L}{\partial x_i}$ ，进而得到 $\frac{\partial y_i}{\partial x_i}$ 。

为了便于实现，你需要确保每个中间梯度的推导都尽可能简化。

完成上述推导后，在函数 `batchnorm_backward_alt` 中实现简化的批标准化反向传播，并通过运行下面的代码比较这两个实现。你的两个实现应计算出几乎相同的结果，但这个替代实现（指 `batchnorm_backward_alt`）应该会稍快一些。

```
[10]: np.random.seed(231) # 设置随机种子，确保结果可复现
N, D = 100, 500 # N 为样本数，D 为特征维度
x = 5 * np.random.randn(N, D) + 12 # 生成随机输入数据（均值约为 12，标准差约为 5）
gamma = np.random.randn(D) # 随机初始化缩放参数 gamma
beta = np.random.randn(D) # 随机初始化偏移参数 beta
dout = np.random.randn(N, D) # 生成随机的输出梯度

bn_param = {'mode': 'train'} # 批标准化参数，设置为训练模式
out, cache = batchnorm_forward(x, gamma, beta, bn_param) # 执行批标准化前向传播，
获取输出和缓存
```



```

# 计算两种反向传播实现的运行时间并比较结果
t1 = time.time() # 记录第一个反向传播开始时间
dx1, dgamma1, dbeta1 = batchnorm_backward(dout, cache) # 使用第一种实现计算反向传播
t2 = time.time() # 记录第一个反向传播结束时间
dx2, dgamma2, dbeta2 = batchnorm_backward_alt(dout, cache) # 使用替代实现计算反向传播
t3 = time.time() # 记录第二个反向传播结束时间

# 打印两种实现的梯度差异和速度提升
print('dx 差异: ', rel_error(dx1, dx2))
print('dgamma 差异: ', rel_error(dgamma1, dgamma2))
print('dbeta 差异: ', rel_error(dbeta1, dbeta2))
print('速度提升: %.2fx' % ((t2 - t1) / (t3 - t2))) # 计算并打印替代实现相对于原实现的速度提升倍数

```

dx 差异: 5.964155941709756e-13

dgamma 差异: 0.0

dbeta 差异: 0.0

速度提升: 0.00x

5 带批标准化的全连接网络

现在你已经有了可用的批标准化实现，回到 `cs231n/classifiers/fc_net.py` 文件中的 `FullyConnectedNet` 类。回想一下，你在作业 1 中实现了该网络的初始化、前向传播和反向传播。将该实现复制到这里，并修改它以融入批标准化。

具体来说，当构造函数中的 `normalization` 参数设置为 "batchnorm" 时，你应该在每个 ReLU 非线性激活函数之前插入一个批标准化层。网络最后一层的输出不应进行标准化。完成后，运行下面的代码对你的实现进行梯度检验。

提示：定义一个类似于 `cs231n/layer_utils.py` 文件中的额外辅助层可能会很有用。

```

[11]: from cs231n.classifiers.fc_net import * # 从 fc_net 模块导入所有内容 (包含
      FullyConnectedNet 类)
      from cs231n.gradient_check import * # 从 gradient_check 模块导入所有内容 (包含梯
      度检验函数)

```



```

np.random.seed(231) # 设置随机种子, 确保结果可复现
N, D, H1, H2, C = 2, 15, 20, 30, 10 # 定义变量: N 为样本数, D 为输入维度, H1、H2
为隐藏层维度, C 为类别数
X = np.random.randn(N, D) # 生成随机输入数据
y = np.random.randint(C, size=(N,)) # 生成随机标签 (0 到 C-1 之间的整数)

# 你应该会看到: W 的梯度相对误差在 1e-4~1e-10 之间,
# b 的梯度相对误差在 1e-08~1e-10 之间,
# beta 和 gamma 的梯度相对误差在 1e-08~1e-09 之间
for reg in [0, 3.14]: # 对两种正则化系数进行测试
    print('使用 reg = ', reg, '进行检验')
    # 初始化带批标准化的全连接网络
    model = FullyConnectedNet([H1, H2], input_dim=D, num_classes=C,
                               reg=reg, weight_scale=5e-2, dtype=np.float64,
                               normalization='batchnorm')

    loss, grads = model.loss(X, y) # 计算损失和梯度
    print('初始损失: ', loss)

    # 对每个参数的梯度进行数值检验
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0] # 定义仅返回损失的函数
        # 用数值方法计算梯度
        grad_num = eval_numerical_gradient(f, model.params[name], verbose=False,
        ↪h=1e-5)
        # 打印参数的梯度相对误差
        print('%s 相对误差: %.2e' % (name, rel_error(grad_num, grads[name])))
    if reg == 0: print() # 当 reg 为 0 时, 打印一个空行分隔结果

```

使用 reg = 0 进行检验

初始损失: 2.2611955101340957

W1 相对误差: 1.10e-04

W2 相对误差: 5.65e-06

W3 相对误差: 4.47e-10

b1 相对误差: 2.22e-08

b2 相对误差: 2.22e-08

b3 相对误差: 1.77e-10
beta1 相对误差: 7.33e-09
beta2 相对误差: 1.89e-09
gamma1 相对误差: 7.47e-09
gamma2 相对误差: 3.35e-09

使用 `reg = 3.14` 进行检验

初始损失: 6.996533220108303

W1 相对误差: 1.98e-06
W2 相对误差: 2.28e-06
W3 相对误差: 1.11e-08
b1 相对误差: 2.78e-09
b2 相对误差: 2.22e-08
b3 相对误差: 1.73e-10
beta1 相对误差: 6.65e-09
beta2 相对误差: 3.48e-09
gamma1 相对误差: 5.94e-09
gamma2 相对误差: 4.67e-09

6 深度网络中的批标准化

运行下面的代码，在 1000 个训练样本的子集上分别训练一个六层网络（包含批标准化和不包含批标准化两种情况）。

```
[12]: np.random.seed(231) # 设置随机种子，确保结果可复现

# 尝试用批标准化训练一个非常深的网络
hidden_dims = [100, 100, 100, 100, 100] # 隐藏层维度列表（5 个隐藏层，每个 100 维）

num_train = 1000 # 训练样本数量
small_data = { # 构建小规模数据集
    'X_train': data['X_train'][:num_train], # 取前 1000 个训练样本
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'], # 验证集保持不变
    'y_val': data['y_val'],
}
```

```

weight_scale = 2e-2 # 权重初始化的缩放因子
# 初始化带批标准化的全连接网络
bn_model = FullyConnectedNet(hidden_dims, weight_scale=weight_scale,
    ↪normalization='batchnorm')
# 初始化不带批标准化的全连接网络 (作为对比)
model = FullyConnectedNet(hidden_dims, weight_scale=weight_scale,
    ↪normalization=None)

print('带批标准化的求解器: ')
# 配置并训练带批标准化的模型
bn_solver = Solver(bn_model, small_data,
    num_epochs=10, # 训练 10 个 epoch
    batch_size=50, # 批大小为 50
    update_rule='adam', # 使用 Adam 优化器
    optim_config={
        'learning_rate': 1e-3, # 学习率为 0.001
    },
    verbose=True, # 打印详细信息
    print_every=20) # 每 20 个迭代打印一次信息
bn_solver.train() # 开始训练

print('\n不带批标准化的求解器: ')
# 配置并训练不带批标准化的模型 (参数与上面保持一致, 便于对比)
solver = Solver(model, small_data,
    num_epochs=10, batch_size=50,
    update_rule='adam',
    optim_config={
        'learning_rate': 1e-3,
    },
    verbose=True, print_every=20)
solver.train() # 开始训练

```

带批标准化的求解器:

(迭代 1 / 200) 损失: 2.340974

(Epoch 0 / 10) 训练准确率: 0.107000; 验证准确率: 0.115000

(Epoch 1 / 10) 训练准确率: 0.312000; 验证准确率: 0.259000

(迭代 21 / 200) 损失: 2.033403
(Epoch 2 / 10) 训练准确率: 0.425000; 验证准确率: 0.293000
(迭代 41 / 200) 损失: 2.041577
(Epoch 3 / 10) 训练准确率: 0.489000; 验证准确率: 0.313000
(迭代 61 / 200) 损失: 1.662143
(Epoch 4 / 10) 训练准确率: 0.542000; 验证准确率: 0.310000
(迭代 81 / 200) 损失: 1.320723
(Epoch 5 / 10) 训练准确率: 0.601000; 验证准确率: 0.301000
(迭代 101 / 200) 损失: 1.355595
(Epoch 6 / 10) 训练准确率: 0.669000; 验证准确率: 0.312000
(迭代 121 / 200) 损失: 1.128599
(Epoch 7 / 10) 训练准确率: 0.674000; 验证准确率: 0.328000
(迭代 141 / 200) 损失: 1.142574
(Epoch 8 / 10) 训练准确率: 0.741000; 验证准确率: 0.324000
(迭代 161 / 200) 损失: 0.714813
(Epoch 9 / 10) 训练准确率: 0.755000; 验证准确率: 0.312000
(迭代 181 / 200) 损失: 1.069392
(Epoch 10 / 10) 训练准确率: 0.775000; 验证准确率: 0.314000

不带批标准化的求解器:

(迭代 1 / 200) 损失: 2.302332
(Epoch 0 / 10) 训练准确率: 0.128000; 验证准确率: 0.132000
(Epoch 1 / 10) 训练准确率: 0.264000; 验证准确率: 0.226000
(迭代 21 / 200) 损失: 2.063807
(Epoch 2 / 10) 训练准确率: 0.325000; 验证准确率: 0.268000
(迭代 41 / 200) 损失: 1.855961
(Epoch 3 / 10) 训练准确率: 0.369000; 验证准确率: 0.282000
(迭代 61 / 200) 损失: 1.772146
(Epoch 4 / 10) 训练准确率: 0.400000; 验证准确率: 0.298000
(迭代 81 / 200) 损失: 1.732161
(Epoch 5 / 10) 训练准确率: 0.483000; 验证准确率: 0.326000
(迭代 101 / 200) 损失: 1.583574
(Epoch 6 / 10) 训练准确率: 0.494000; 验证准确率: 0.324000
(迭代 121 / 200) 损失: 1.599862
(Epoch 7 / 10) 训练准确率: 0.553000; 验证准确率: 0.310000
(迭代 141 / 200) 损失: 1.335549
(Epoch 8 / 10) 训练准确率: 0.565000; 验证准确率: 0.325000

(迭代 161 / 200) 损失: 1.167843

(Epoch 9 / 10) 训练准确率: 0.648000; 验证准确率: 0.339000

(迭代 181 / 200) 损失: 0.874595

(Epoch 10 / 10) 训练准确率: 0.681000; 验证准确率: 0.322000

运行下面的代码, 可视化上面训练的两个网络的结果。你会发现, 使用批标准化能帮助网络更快收敛。

```
[15]: def plot_training_history(title, label, baseline, bn_solvers, plot_fn,
    ↪bl_marker='.', bn_marker='.', labels=None):
    """ 用于绘制训练历史的工具函数 """
    plt.title(title) # 设置图表标题
    plt.xlabel(label) # 设置 x 轴标签

    # 为带批标准化的求解器生成要绘制的数据
    bn_plots = [plot_fn(bn_solver) for bn_solver in bn_solvers]
    # 为基准求解器 (不带批标准化) 生成要绘制的数据
    bl_plot = plot_fn(baseline)

    num_bn = len(bn_plots)
    for i in range(num_bn):
        label = 'with_norm' # 带批标准化的标签
        if labels is not None:
            label += str(labels[i]) # 如果提供了标签列表, 添加额外标识
        # 绘制带批标准化的曲线
        plt.plot(bn_plots[i], bn_marker, label=label)

    label = 'baseline' # 基准 (不带批标准化) 的标签
    if labels is not None:
        label += str(labels[0])
    # 绘制基准曲线
    plt.plot(bl_plot, bl_marker, label=label)

    # 设置图例位置和列数
    plt.legend(loc='lower center', ncol=num_bn + 1)
```

```

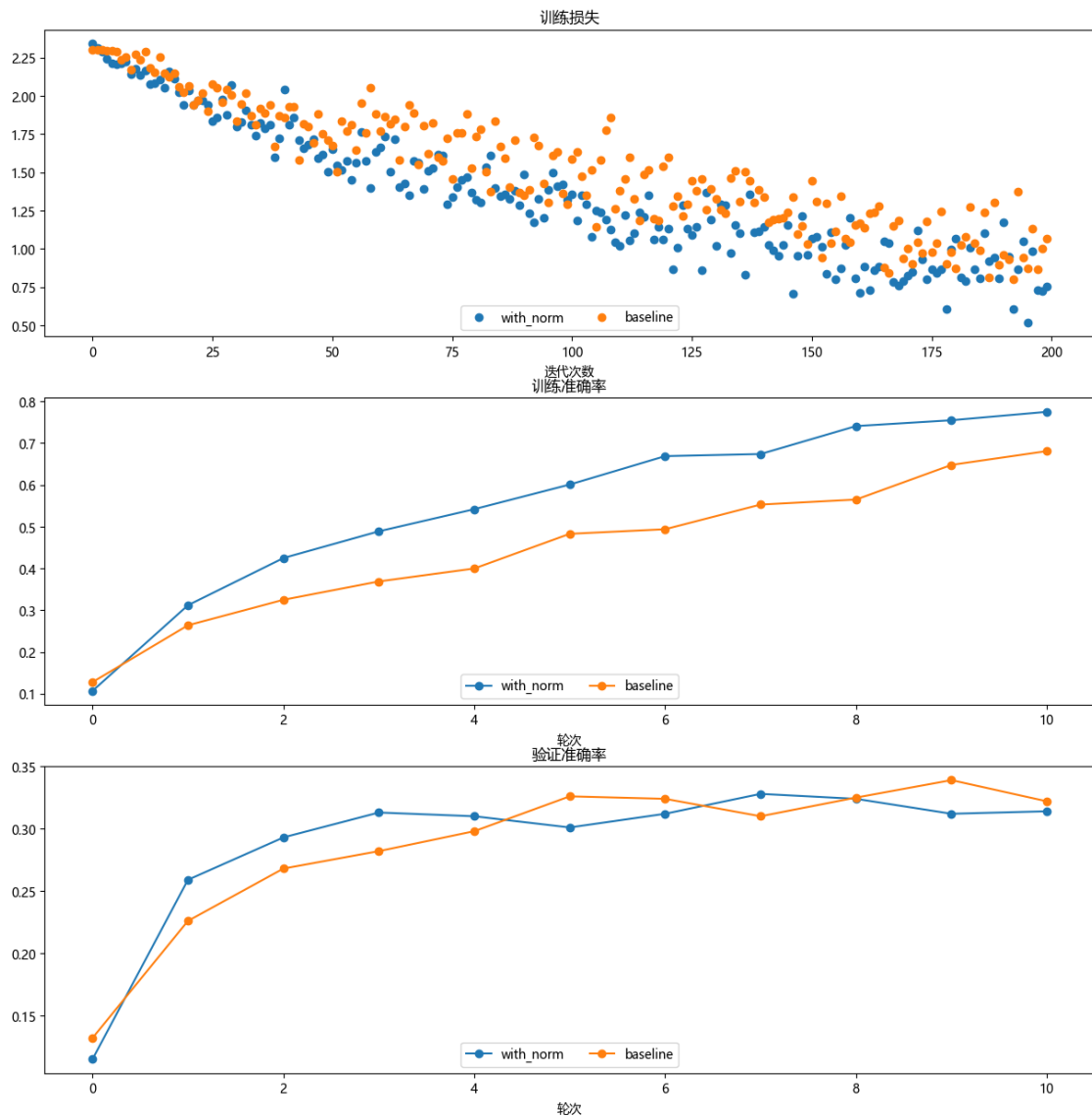
# 创建 3 行 1 列的子图布局, 绘制第一个子图 (训练损失)
plt.subplot(3, 1, 1)
plot_training_history('训练损失', '迭代次数', solver, [bn_solver],
                      lambda x: x.loss_history, bl_marker='o', bn_marker='o')

# 绘制第二个子图 (训练准确率)
plt.subplot(3, 1, 2)
plot_training_history('训练准确率', '轮次', solver, [bn_solver],
                      lambda x: x.train_acc_history, bl_marker='-o',
                      bn_marker='-o')

# 绘制第三个子图 (验证准确率)
plt.subplot(3, 1, 3)
plot_training_history('验证准确率', '轮次', solver, [bn_solver],
                      lambda x: x.val_acc_history, bl_marker='-o',
                      bn_marker='-o')

# 设置整个图表的尺寸
plt.gcf().set_size_inches(15, 15)
# 显示图表
plt.show()

```



7 批标准化与初始化

我们现在将进行一个小实验，研究批标准化与权重初始化之间的相互作用。

第一个单元格将使用不同的权重初始化缩放因子，分别训练带批标准化和不带批标准化的八层网络。第二个单元格将绘制训练准确率、验证集准确率以及训练损失随权重初始化缩放因子变化的曲线。


```

[16]: np.random.seed(231) # 设置随机种子, 确保结果可复现

# 尝试用批标准化训练一个非常深的网络
hidden_dims = [50, 50, 50, 50, 50, 50, 50] # 隐藏层维度列表 (7 个隐藏层, 每个 50 维)
num_train = 1000 # 训练样本数量
small_data = { # 构建小规模数据集
    'X_train': data['X_train'][:num_train], # 取前 1000 个训练样本
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'], # 验证集保持不变
    'y_val': data['y_val'],
}

# 用于存储不同权重缩放因子对应的求解器
bn_solvers_ws = {} # 带批标准化的求解器字典
solvers_ws = {} # 不带批标准化的求解器字典
# 生成 20 个权重初始化缩放因子, 范围从 1e-4 到 1 (对数均匀分布)
weight_scales = np.logspace(-4, 0, num=20)

# 遍历每个权重缩放因子, 分别训练带和不带批标准化的模型
for i, weight_scale in enumerate(weight_scales):
    print('正在运行权重缩放因子 %d / %d' % (i + 1, len(weight_scales)))
    # 初始化带批标准化的全连接网络
    bn_model = FullyConnectedNet(hidden_dims, weight_scale=weight_scale,
    ↪normalization='batchnorm')
    # 初始化不带批标准化的全连接网络
    model = FullyConnectedNet(hidden_dims, weight_scale=weight_scale,
    ↪normalization=None)

    # 训练带批标准化的模型
    bn_solver = Solver(bn_model, small_data,
        num_epochs=10, # 训练 10 个 epoch
        batch_size=50, # 批大小为 50
        update_rule='adam', # 使用 Adam 优化器
        optim_config={
            'learning_rate': 1e-3, # 学习率为 0.001

```

```

    },
    verbose=False, # 不打印详细信息
    print_every=200) # 每 200 个迭代打印一次信息
bn_solver.train() # 开始训练
bn_solvers_ws[weight_scale] = bn_solver # 存储求解器

# 训练不带批标准化的模型（参数与带批标准化的模型保持一致）
solver = Solver(model, small_data,
                num_epochs=10, batch_size=50,
                update_rule='adam',
                optim_config={
                    'learning_rate': 1e-3,
                },
                verbose=False, print_every=200)
solver.train() # 开始训练
solvers_ws[weight_scale] = solver # 存储求解器

```

正在运行权重缩放因子 1 / 20
 正在运行权重缩放因子 2 / 20
 正在运行权重缩放因子 3 / 20
 正在运行权重缩放因子 4 / 20
 正在运行权重缩放因子 5 / 20
 正在运行权重缩放因子 6 / 20
 正在运行权重缩放因子 7 / 20
 正在运行权重缩放因子 8 / 20
 正在运行权重缩放因子 9 / 20
 正在运行权重缩放因子 10 / 20
 正在运行权重缩放因子 11 / 20
 正在运行权重缩放因子 12 / 20
 正在运行权重缩放因子 13 / 20
 正在运行权重缩放因子 14 / 20
 正在运行权重缩放因子 15 / 20
 正在运行权重缩放因子 16 / 20

e:\学习\计算机视觉\assignment-Chinese\assignment2\cs231n\layers.py:138: ␣

␣RuntimeWarning:

divide by zero encountered in log

```
correct_logprobs = -np.log(probs[np.arange(N), y])
```

正在运行权重缩放因子 17 / 20

正在运行权重缩放因子 18 / 20

正在运行权重缩放因子 19 / 20

正在运行权重缩放因子 20 / 20

```
[17]: # 绘制权重缩放因子实验的结果
best_train_accs, bn_best_train_accs = [], [] # 存储最佳训练准确率（分别对应无批标
准化和有批标准化的模型）
best_val_accs, bn_best_val_accs = [], [] # 存储最佳验证准确率（分别对应无批标
准化和有批标准化的模型）
final_train_loss, bn_final_train_loss = [], [] # 存储最终训练损失（分别对应无批标
准化和有批标准化的模型）

# 遍历每个权重缩放因子，提取并保存相关指标
for ws in weight_scales:
    # 无批标准化模型的最佳训练准确率
    best_train_accs.append(max(solvers_ws[ws].train_acc_history))
    # 有批标准化模型的最佳训练准确率
    bn_best_train_accs.append(max(bn_solvers_ws[ws].train_acc_history))

    # 无批标准化模型的最佳验证准确率
    best_val_accs.append(max(solvers_ws[ws].val_acc_history))
    # 有批标准化模型的最佳验证准确率
    bn_best_val_accs.append(max(bn_solvers_ws[ws].val_acc_history))

    # 无批标准化模型最后 100 个损失的平均值（作为最终训练损失）
    final_train_loss.append(np.mean(solvers_ws[ws].loss_history[-100:]))
    # 有批标准化模型最后 100 个损失的平均值
    bn_final_train_loss.append(np.mean(bn_solvers_ws[ws].loss_history[-100:]))

# 绘制第一个子图：最佳验证准确率与权重初始化缩放因子的关系
plt.subplot(3, 1, 1)
plt.title('最佳验证准确率 vs. 权重初始化缩放因子')
plt.xlabel('权重初始化缩放因子')
plt.ylabel('最佳验证准确率')
```

```

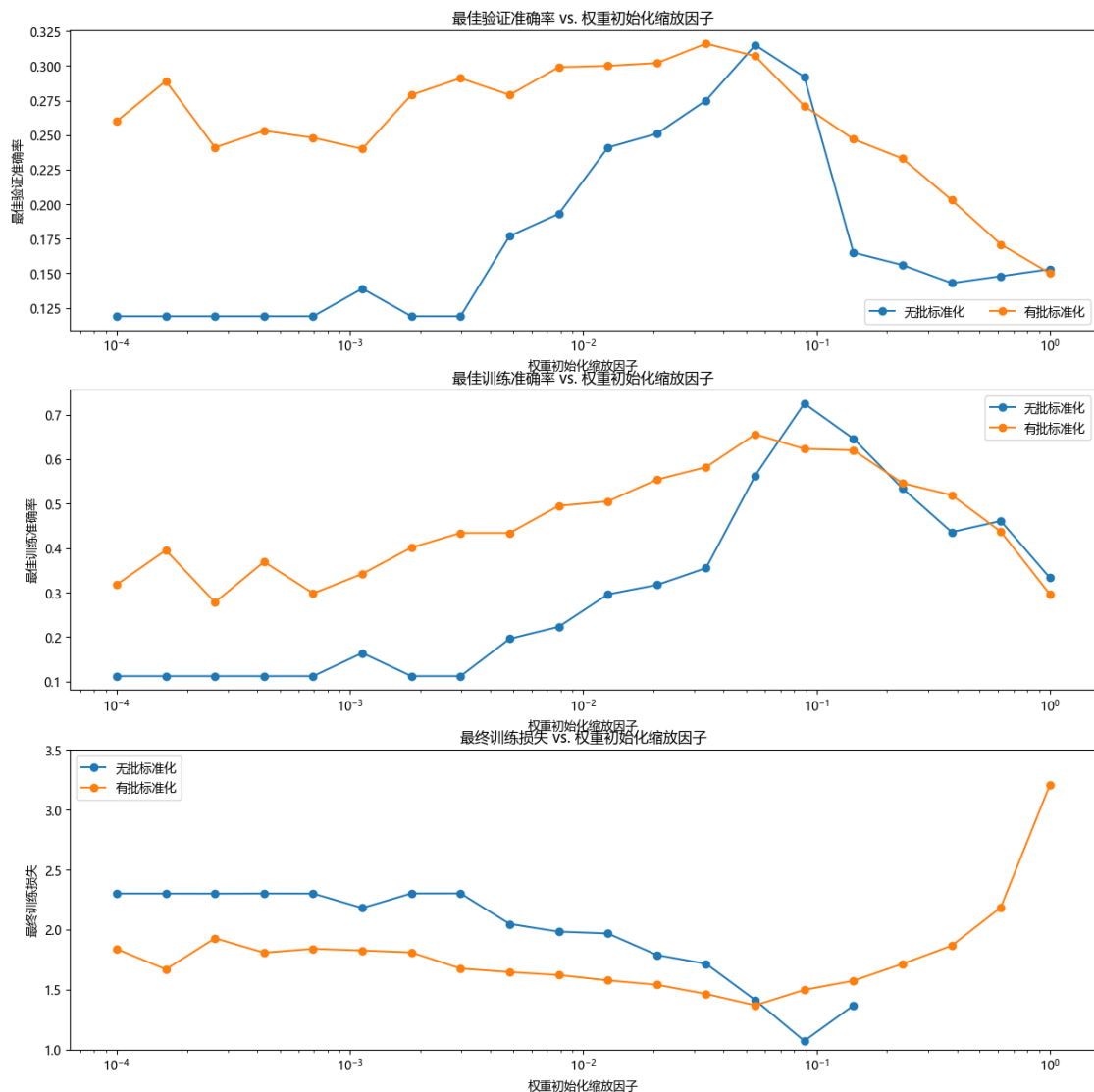
# 用对数坐标绘制无批标准化模型的曲线
plt.semilogx(weight_scales, best_val_accs, '-o', label='无批标准化')
# 用对数坐标绘制有批标准化模型的曲线
plt.semilogx(weight_scales, bn_best_val_accs, '-o', label='有批标准化')
plt.legend(ncol=2, loc='lower right') # 设置图例, 2 列, 位于右下角

# 绘制第二个子图: 最佳训练准确率与权重初始化缩放因子的关系
plt.subplot(3, 1, 2)
plt.title('最佳训练准确率 vs. 权重初始化缩放因子')
plt.xlabel('权重初始化缩放因子')
plt.ylabel('最佳训练准确率')
plt.semilogx(weight_scales, best_train_accs, '-o', label='无批标准化')
plt.semilogx(weight_scales, bn_best_train_accs, '-o', label='有批标准化')
plt.legend() # 设置图例

# 绘制第三个子图: 最终训练损失与权重初始化缩放因子的关系
plt.subplot(3, 1, 3)
plt.title('最终训练损失 vs. 权重初始化缩放因子')
plt.xlabel('权重初始化缩放因子')
plt.ylabel('最终训练损失')
plt.semilogx(weight_scales, final_train_loss, '-o', label='无批标准化')
plt.semilogx(weight_scales, bn_final_train_loss, '-o', label='有批标准化')
plt.legend() # 设置图例
plt.gca().set_ylim(1.0, 3.5) # 设置 y 轴范围

plt.gcf().set_size_inches(15, 15) # 设置整个图表的尺寸
plt.show() # 显示图表

```



7.1 内联问题 1:

描述这个实验的结果。权重初始化缩放因子对带有/不带有批标准化的模型的影响有何不同？为什么会这样？

7.2 答案:

带批标准化的模型对权重初始化缩放因子不敏感：不同的初始化缩放下训练/验证曲线差异很小，模型能稳定收敛。

不带批标准化的模型对缩放因子很敏感：缩放太小会导致激活和梯度极小、学习很慢或欠拟合；缩

放太大会使激活分布发散/饱和、梯度不稳定或训练失败。

原因：批标准化在每层把输入变为近似零均值单位方差（并用可学习的 γ/β 恢复尺度和平移），从而消除了权重初始尺度对层输入分布和梯度尺度的直接影响，稳定了训练（降低“内部协变量偏移”），因此减轻了对精细权重初始化的依赖。

8 批标准化与批大小

我们现在将进行一个小实验，研究批标准化与批大小之间的相互作用。

第一个单元格将使用不同的批大小，分别训练带批标准化和不带批标准化的六层网络。第二个单元格将绘制训练过程中的训练准确率和验证集准确率随时间的变化曲线。

```
[18]: def run_batchsize_experiments(normalization_mode):
    np.random.seed(231) # 设置随机种子，确保结果可复现

    # 尝试用批标准化训练一个非常深的网络
    hidden_dims = [100, 100, 100, 100, 100] # 隐藏层维度列表（5 个隐藏层，每个 100 维）
    num_train = 1000 # 训练样本数量
    small_data = { # 构建小规模数据集
        'X_train': data['X_train'][:num_train], # 取前 1000 个训练样本
        'y_train': data['y_train'][:num_train],
        'X_val': data['X_val'], # 验证集保持不变
        'y_val': data['y_val'],
    }
    n_epochs = 10 # 训练轮数
    weight_scale = 2e-2 # 权重初始化的缩放因子
    batch_sizes = [5, 10, 50] # 要测试的批大小列表
    lr = 10 ** (-3.5) # 学习率
    solver_bsize = batch_sizes[0] # 基准模型使用的批大小（取列表中的第一个值）

    print('无标准化: 批大小 = ', solver_bsize)
    # 初始化无批标准化的全连接网络
    model = FullyConnectedNet(hidden_dims, weight_scale=weight_scale,
    ↪ normalization=None)
    # 训练无批标准化的模型
    solver = Solver(model, small_data,
```

```

        num_epochs=n_epochs, batch_size=solver_bsize,
        update_rule='adam', # 使用 Adam 优化器
        optim_config={
            'learning_rate': lr, # 学习率设置
        },
        verbose=False) # 不打印详细信息
solver.train() # 开始训练

bn_solvers = [] # 存储不同批大小下带批标准化的求解器
for i in range(len(batch_sizes)):
    b_size = batch_sizes[i] # 当前批大小
    print('有标准化: 批大小 = ', b_size)
    # 初始化带批标准化的全连接网络
    bn_model = FullyConnectedNet(hidden_dims, weight_scale=weight_scale,
    ↪normalization=normalization_mode)
    # 训练带批标准化的模型
    bn_solver = Solver(bn_model, small_data,
        num_epochs=n_epochs, batch_size=b_size,
        update_rule='adam',
        optim_config={
            'learning_rate': lr,
        },
        verbose=False)
    bn_solver.train() # 开始训练
    bn_solvers.append(bn_solver) # 将求解器添加到列表

    return bn_solvers, solver, batch_sizes # 返回带批标准化的求解器、基准求解器和
批大小列表

batch_sizes = [5, 10, 50] # 定义要测试的批大小
# 运行批大小实验, 获取结果 (带批标准化的求解器、基准求解器和批大小)
bn_solvers_bsize, solver_bsize, batch_sizes =
    ↪run_batchsize_experiments('batchnorm')

```

无标准化: 批大小 = 5

有标准化: 批大小 = 5

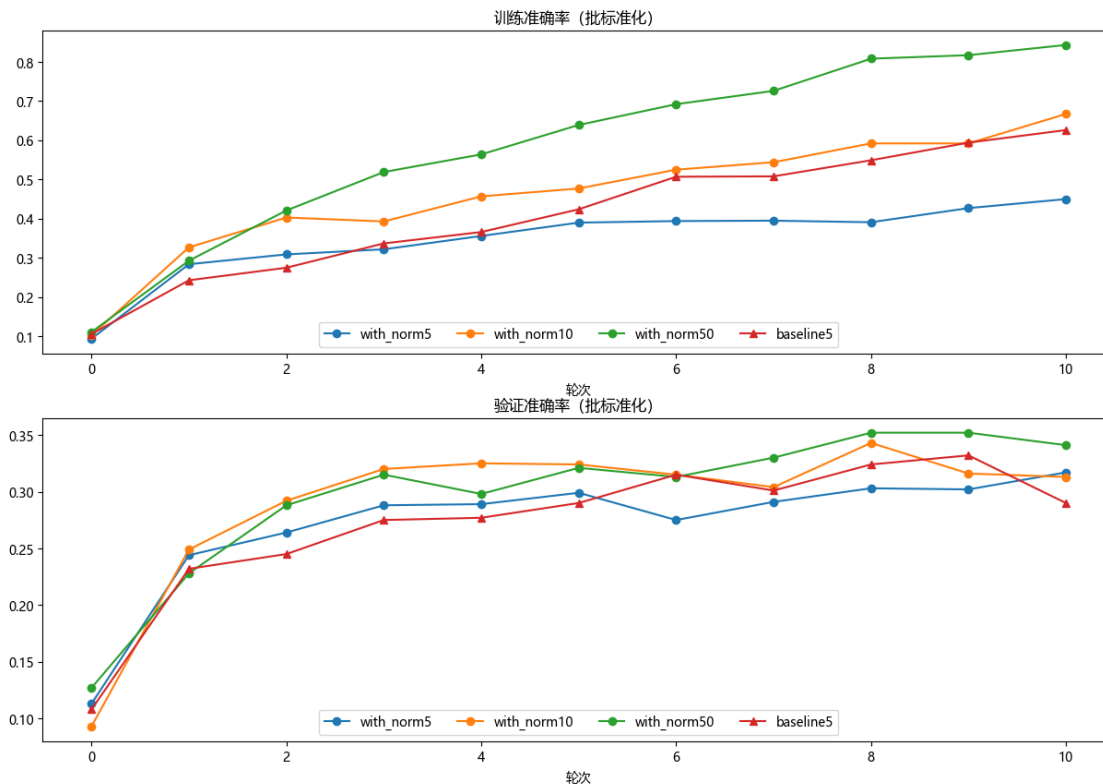
有标准化: 批大小 = 10

有标准化: 批大小 = 50

```
[19]: # 绘制第一个子图: 批标准化实验中的训练准确率
plt.subplot(2, 1, 1)
plot_training_history('训练准确率 (批标准化)', '轮次', solver_bsize,
    ↪bn_solvers_bsize,
    lambda x: x.train_acc_history, bl_marker='^-',
    ↪bn_marker='-o', labels=batch_sizes)

# 绘制第二个子图: 批标准化实验中的验证准确率
plt.subplot(2, 1, 2)
plot_training_history('验证准确率 (批标准化)', '轮次', solver_bsize,
    ↪bn_solvers_bsize,
    lambda x: x.val_acc_history, bl_marker='^-',
    ↪bn_marker='-o', labels=batch_sizes)

# 设置整个图表的尺寸
plt.gcf().set_size_inches(15, 10)
# 显示图表
plt.show()
```

8.1 内联问题 2:

描述这个实验的结果。这对于批标准化和批大小之间的关系意味着什么？为什么会观察到这种关系？

8.2 答案:

实验结果显示：在使用批标准化时，较大的批大小（如 50）在训练阶段收敛更快、训练准确率更高；较小的批次（如 5、10）训练准确率较低。验证集准确率的差异要小得多，但较大批次通常也能达到或略高于小批次的验证效果。

批标准化依赖于每个 mini-batch 的均值和方差估计，批大小越大，这些统计量的估计越稳定、噪声越小，从而使标准化更有效并稳定梯度更新；批大小太小会导致统计量噪声大、归一化不稳定，从而削弱批标准化的优点。没有批标准化的基线在相同小批下表现更差，进一步说明了批标准化能改善训练稳定性，但其效果受批大小限制。

若受限于小批量，可考虑使用层标准化或组归一化等不依赖大批的归一化方法，或调整学习率/优化器等以缓解小批带来的噪声影响。

9 层标准化

批标准化已被证明能有效简化网络的训练过程，但它对批大小的依赖性使其在一些复杂网络中实用性降低——这些网络由于硬件限制，对输入批大小有上限。

为缓解这一问题，人们提出了多种批标准化的替代方法，层标准化便是其中之一 [2]。与在批次上进行标准化不同，层标准化是在特征上进行标准化。换句话说，使用层标准化时，每个数据点对应的特征向量会根据该特征向量内所有元素的总和进行标准化。

[2] Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton. “Layer Normalization.” *stat 1050* (2016): 21.

9.1 内联问题 3:

以下哪些数据预处理步骤类似于批标准化，哪些类似于层标准化？

1. 缩放数据集中的每张图像，使图像内每一行像素的 RGB 通道总和为 1。
2. 缩放数据集中的每张图像，使图像内所有像素的 RGB 通道总和为 1。
3. 从数据集中的每张图像中减去数据集的平均图像。
4. 根据给定的阈值将所有 RGB 值设为 0 或 1。

9.2 答案:

1. 层标准化：因为这是在单个样本的内部按行/通道对特征进行归一化，属于对样本内部特征维度的操作。
2. 层标准化：对整张图像的所有像素/通道进行归一化，仍然是对单个样本的特征维度进行处理。
3. 批标准化：从整个数据集（或大批次）计算并减去平均图像，相当于使用跨样本的统计量来中心化每个空间/通道特征。
4. 两者都不相似：按阈值离散化像素值是非线性量化/二值化，不属于按均值/方差的线性归一化，因此既不等价于批标准化也不等价于层标准化。

10 层标准化：实现

现在你要实现层标准化。这一步应该相对简单，因为从概念上讲，它的实现几乎与批标准化相同。不过一个显著的区别是，对于层标准化，我们不需要跟踪移动矩（moving moments），而且测试阶段与训练阶段完全相同，即均值和方差是直接针对每个数据点计算的。

你需要做以下事情：

- 在 `cs231n/layers.py` 中，在 `layernorm_forward` 函数里实现层标准化的前向传播。

运行下面的单元格来检查你的结果。* 在 `cs231n/layers.py` 中, 在 `layernorm_backward` 函数里实现层标准化的反向传播。

运行下面的第二个单元格来检查你的结果。* 修改 `cs231n/classifiers/fc_net.py`, 为 `FullyConnectedNet` 添加层标准化功能。当构造函数中的 `normalization` 参数设置为 "layernorm" 时, 你应该在每个 `ReLU` 非线性激活函数之前插入一个层标准化层。

运行下面的第三个单元格, 在层标准化上运行批大小实验。

```
[20]: # 通过检查特征在层标准化前后的均值和方差, 验证训练时的前向传播
# 模拟两层网络的前向传播过程
np.random.seed(231) # 设置随机种子, 确保结果可复现
N, D1, D2, D3 = 4, 50, 60, 3 # 定义变量: N 为样本数, D1、D2 为中间层维度, D3 为输出维度

X = np.random.randn(N, D1) # 生成随机输入数据
W1 = np.random.randn(D1, D2) # 第一层权重
W2 = np.random.randn(D2, D3) # 第二层权重
a = np.maximum(0, X.dot(W1)).dot(W2) # 计算经过两层网络 (含 ReLU 激活) 后的输出

print('层标准化前: ')
print_mean_std(a, axis=1) # 打印标准化前特征的均值和标准差 (按样本维度计算)

gamma = np.ones(D3) # 缩放参数 gamma 初始化为 1
beta = np.zeros(D3) # 偏移参数 beta 初始化为 0

# 标准化后, 均值应接近 0, 标准差应接近 1
print('层标准化后 (gamma=1, beta=0) ')
a_norm, _ = layernorm_forward(a, gamma, beta, {'mode': 'train'}) # 执行层标准化前向传播
print_mean_std(a_norm, axis=1) # 打印标准化后特征的均值和标准差

gamma = np.asarray([3.0, 3.0, 3.0]) # 缩放参数 gamma 设为 3
beta = np.asarray([5.0, 5.0, 5.0]) # 偏移参数 beta 设为 5

# 此时均值应接近 beta, 标准差应接近 gamma
print('层标准化后 (gamma=', gamma, ', beta=', beta, ') ')
a_norm, _ = layernorm_forward(a, gamma, beta, {'mode': 'train'}) # 执行层标准化前向传播
```

```
print_mean_std(a_norm, axis=1) # 打印标准化后特征的均值和标准差
```

层标准化前:

均值: [-59.06673243 -47.60782686 -43.31137368 -26.40991744]

标准差: [10.07429373 28.39478981 35.28360729 4.01831507]

层标准化后 (gamma=1, beta=0)

均值: [4.81096644e-16 0.00000000e+00 0.00000000e+00 -2.96059473e-16]

标准差: [0.99999995 0.99999999 1. 0.99999969]

层标准化后 (gamma= [3. 3. 3.] , beta= [5. 5. 5.])

均值: [5. 5. 5. 5.]

标准差: [2.99999985 2.99999998 2.99999999 2.99999907]

```
[21]: # 梯度检验层标准化的反向传播
np.random.seed(231) # 设置随机种子, 确保结果可复现
N, D = 4, 5 # N 为样本数, D 为特征维度
x = 5 * np.random.randn(N, D) + 12 # 生成随机输入数据 (均值为 12, 标准差为 5)
gamma = np.random.randn(D) # 生成随机缩放参数 gamma
beta = np.random.randn(D) # 生成随机偏移参数 beta
dout = np.random.randn(N, D) # 生成随机的上游梯度

ln_param = {} # 层标准化的参数字典
# 定义仅以 x 为参数的层标准化前向传播函数 (用于数值梯度计算)
fx = lambda x: layernorm_forward(x, gamma, beta, ln_param)[0]
# 定义仅以 gamma 为参数的层标准化前向传播函数
fg = lambda a: layernorm_forward(x, a, beta, ln_param)[0]
# 定义仅以 beta 为参数的层标准化前向传播函数
fb = lambda b: layernorm_forward(x, gamma, b, ln_param)[0]

# 用数值方法计算 x 的梯度
dx_num = eval_numerical_gradient_array(fx, x, dout)
# 用数值方法计算 gamma 的梯度
da_num = eval_numerical_gradient_array(fg, gamma.copy(), dout)
# 用数值方法计算 beta 的梯度
db_num = eval_numerical_gradient_array(fb, beta.copy(), dout)
```

```

# 执行层标准化的前向传播，获取缓存
_, cache = layernorm_forward(x, gamma, beta, ln_param)
# 执行层标准化的反向传播，计算解析梯度
dx, dgamma, dbeta = layernorm_backward(dout, cache)

# 你应该会看到相对误差在  $1e-12$  到  $1e-8$  之间
print('dx 误差: ', rel_error(dx_num, dx)) # 打印  $x$  的解析梯度与数值梯度的相对误差
print('dgamma 误差: ', rel_error(da_num, dgamma)) # 打印  $\gamma$  的解析梯度与数值梯度的相对误差
print('dbeta 误差: ', rel_error(db_num, dbeta)) # 打印  $\beta$  的解析梯度与数值梯度的相对误差

```

```

dx 误差: 1.433615146847572e-09
dgamma 误差: 4.519489546032799e-12
dbeta 误差: 2.276445013433725e-12

```

11 层标准化与批大小

我们现在将用层标准化代替批标准化来运行之前的批大小实验。与之前的实验相比，你会发现批大小对训练过程的影响明显更小！

```

[22]: # 运行批大小实验，使用层标准化并获取结果（带层标准化的求解器、基准求解器和批大小）
ln_solvers_bsize, solver_bsize, batch_sizes = \
    ↪run_batchsize_experiments('layernorm')

# 绘制第一个子图：层标准化实验中的训练准确率
plt.subplot(2, 1, 1)
plot_training_history('训练准确率（层标准化）', '轮次', solver_bsize, \
    ↪ln_solvers_bsize,
                        lambda x: x.train_acc_history, bl_marker='^-', \
    ↪bn_marker='-o', labels=batch_sizes)

# 绘制第二个子图：层标准化实验中的验证准确率
plt.subplot(2, 1, 2)
plot_training_history('验证准确率（层标准化）', '轮次', solver_bsize, \
    ↪ln_solvers_bsize,

```

```

        lambda x: x.val_acc_history, bl_marker='^-',
        bn_marker='-o', labels=batch_sizes)

# 设置整个图表的尺寸
plt.gcf().set_size_inches(15, 10)
# 显示图表
plt.show()

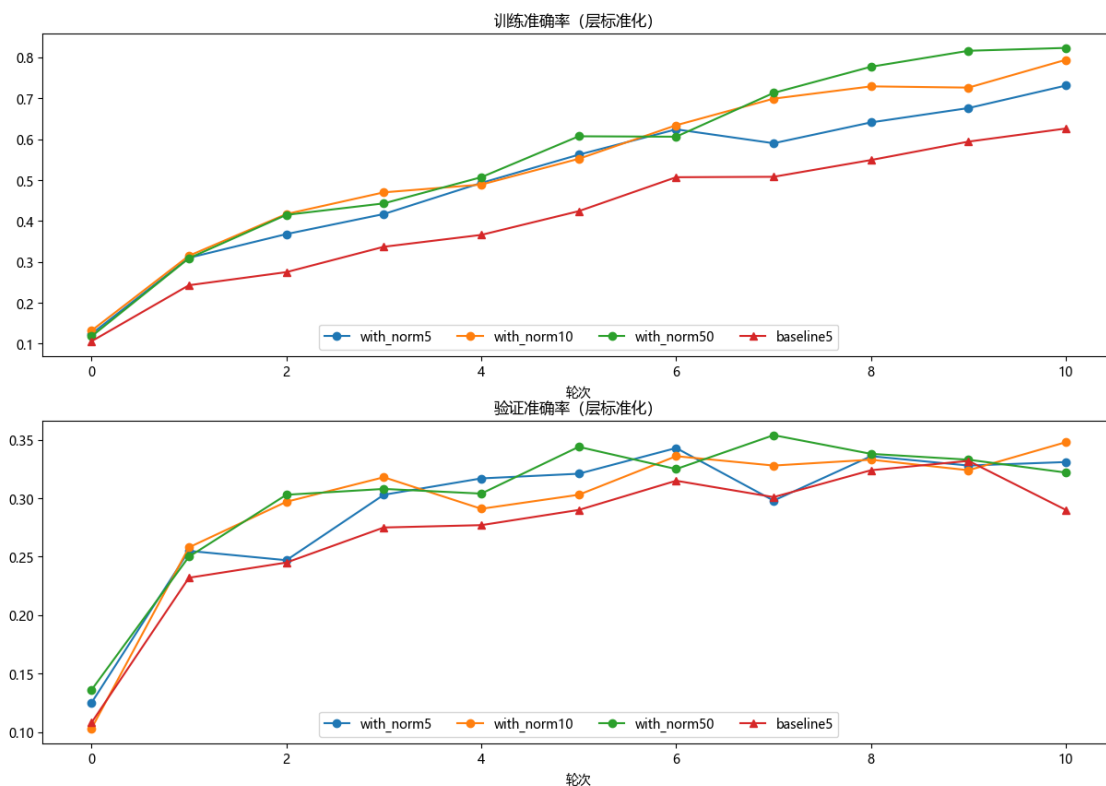
```

无标准化: 批大小 = 5

有标准化: 批大小 = 5

有标准化: 批大小 = 10

有标准化: 批大小 = 50



11.1 内联问题 4:

层标准化在什么情况下可能效果不佳? 为什么?

1. 在极深的网络中使用它

2. 特征维度非常小
3. 有很大的正则化项

11.2 答案:

1. 在极深的网络中: 层标准化在序列模型 (如 Transformer、RNN) 中通常效果很好, 但在某些极深的卷积网络中, 批标准化利用批次统计带来的额外稳定性和正则化效果可能更有优势。因此在极深的卷积结构里, 层标准化的改进可能没有批标准化明显。
2. 特征维度非常小: 层标准化是沿每个样本的特征维度进行归一化, 当特征维度 D 很小时, 样本内的均值/方差估计基数小, 统计量方差大, 归一化会变得不稳定, 效果会下降。
3. 有很大的正则化项: 若使用非常强的正则化 (如过大的权重衰减或对 γ/β 的强约束), 会限制层归一化学习到合适的放缩和平移, 从而削弱其作用; 同时强正则化可能导致欠拟合, 使归一化带来的益处难以体现。